

Race Condition Vulnerabilities

Methodology & Checklist

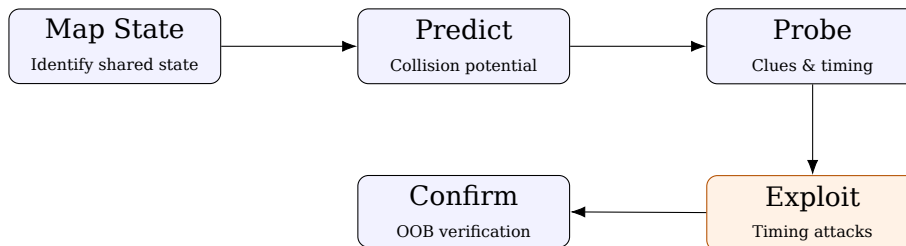
Eyad Islam El-Taher

Contents

1 Methodology Overview	2
2 Phase 1: Predict Potential Collisions	2
2.1 Map the Application State	2
2.2 Types of Race Conditions	2
3 Phase 2: Probe for Clues	3
3.1 Initial Probing	3
3.2 Token/Parameter Fuzzing	3
3.3 Timing Analysis	3
4 Phase 3: Timing Techniques	4
4.1 HTTP/1.1 Last-Byte Synchronization	4
4.2 HTTP/2 Single-Packet Attack	4
4.3 Bypassing Session-Based Locking	4
5 Phase 4: Exploitation	5
5.1 Pattern 1: Limit-Overrun	5
5.2 Pattern 2: Rate-Limit Bypass	5
5.3 Pattern 3: Multi-Endpoint Race	6
5.4 Pattern 4: Single-Endpoint Race	6
5.5 Pattern 5: Partial-Construction Race	7
5.6 Pattern 6: Time-Sensitive Tokens	7
6 Phase 5: Impact Confirmation	8
7 Complete Testing Checklist	8
8 Turbo Intruder Templates	9
8.1 Template 1: Single Request Repeated	9
8.2 Template 2: Two Different Requests Racing	9
8.3 Template 3: Single-Packet Attack (HTTP/2)	10
9 Tools Reference	10
10 Common Pitfalls & How to Avoid Them	10
11 Prevention Recommendations	11
12 References	11

1 Methodology Overview

- **Goal:** Identify, confirm, and exploit race condition vulnerabilities across web applications.
- **Approach:** Systematic mapping → collision prediction → probing → timing control → exploitation → confirmation.
- **Key mindset:** Stop thinking in terms of “one request, one response.” Every endpoint that touches shared state is really asking: *what happens if this exact code path runs twice, at the same instant, for the same record?*



2 Phase 1: Predict Potential Collisions

2.1 Map the Application State

Checklist

- ✓ **Security-critical endpoints:** Endpoints with no real security/business impact are not worth racing
- ✓ **Collision potential:** Need two or more requests that operate on the *same* underlying record
 - Same user session (cookies)
 - Same resource ID (product, order, account)
 - Same shared counter (coupon, balance, attempts)
- ✓ **Look for:**
 - Coupons/discounts and gift cards
 - Balances and financial transactions
 - Login/2FA/OTP attempt counters
 - Password reset and email-change flows
 - Invite/registration systems
 - Any “one-time” action
 - Multi-step checkouts

2.2 Types of Race Conditions

Type	Description	Example
Limit-Overrun	Check-then-act on a counter	Multiple coupon redemptions
Rate-Limit Bypass	Counter check not atomic	Brute-force past lockout
Multi-Endpoint	Two different endpoints race	Add-to-cart vs checkout
Single-Endpoint	Same endpoint, different values	Email change with two addresses
Partial-Construction	Object read mid-creation	Confirm before token stored
Time-Sensitive	Predictable token generation	Timestamp-based reset tokens

3 Phase 2: Probe for Clues

3.1 Initial Probing

Checklist

- ✓ Perform the action normally once, note response and timing
- ✓ Repeat the action and compare: does the token/value change each time?
- ✓ Is the length constant? (suggests fixed-format hash or ID)
- ✓ Send request twice in Repeater sequentially, note response times
- ✓ If every response is delayed/serialized under parallel send → suspect **session-based locking** (PHP native session handler)
- ✓ If session locking is masking the race → retest using **two separate sessions**

3.2 Token/Parameter Fuzzing

Checklist

- ✓ Try malformed/edge-case values on token-comparison endpoints:
 - Missing parameter
 - Empty string (token=)
 - Empty array (token[]=)
 - Null-equivalent values
- ✓ Different error messages reveal how comparison is implemented:
 - Missing parameter vs Incorrect token vs Forbidden
 - Array in error message suggests array was accepted as input

3.3 Timing Analysis

Checklist

- ✓ Benchmark relative response times between requests you plan to race
- ✓ If one is consistently much faster → need deliberate client-side delay
- ✓ Fast request must land *inside* slow request's race window
- ✓ Use:

- **Warm connections:** send a throwaway request first
- **Deliberate delay:** add `sleep()` or use Turbo Intruder's gate control
- **Slow responses:** make remote file respond slowly (for URL-based uploads)

4 Phase 3: Timing Techniques

4.1 HTTP/1.1 Last-Byte Synchronization

Timing Technique

- Send every request in the group **except the final byte**
- Hold each connection open with all requests parked mid-stream
- Release the last byte of every request back-to-back
- All requests reach application layer within microseconds

```
engine.queue(request, gate='race1')  
engine.openGate('race1') # releases withheld final bytes together
```

4.2 HTTP/2 Single-Packet Attack

Timing Technique

- HTTP/2 multiplexes several requests over one TCP connection
- Stuff 20-30 complete requests into a single TCP packet
- All arrive at the server in the same instant
- Removes network jitter entirely
- In Burp: duplicate request 20+ times → *Send group in parallel (single-packet attack)*

4.3 Bypassing Session-Based Locking

Timing Technique

- PHP's native session handler processes one request at a time *per session*
- This serializes requests, masking race conditions
- **Bypass:** Use two different sessions for two racing requests
- Get a fresh session by making a request without a session cookie
- Swap the fresh session/CSRF pair into one of the racing requests

5 Phase 4: Exploitation

5.1 Pattern 1: Limit-Overrun

- Pattern: One-time coupon/discount
1. Send POST /cart/coupon with the discount code
 2. Duplicate into a group of 20+ tabs
 3. Send group **in parallel** (single-packet attack)
 4. Refresh cart — discount applied multiple times
 5. Stack discounts until price under target

Root Cause Pattern

```
if not code.used:
    apply_discount(order)
    code.used = True # Write happens AFTER discount is applied
```

Root Cause (Programming Perspective)

If 20 requests reach `if not code.used` before any reaches the write, all 20 see `used = False` and all 20 apply the discount. No lock/transaction wraps the check-and-set as one atomic unit.

5.2 Pattern 2: Rate-Limit Bypass

Pattern: Brute-force past lockout

1. Submit wrong passwords for a user — confirm lockout after 3 attempts
2. Use Turbo Intruder with single-packet attack
3. Load password wordlist, set username=target
4. Fire all guesses in one race window
5. Watch for 302 redirect (successful login)

Root Cause Pattern

```
attempts = get_failed_attempts(username) # read
if attempts >= 3: return "Locked"
verify_password(username, password) # slow: DB/hash compare
increment_failed_attempts(username) # write happens LAST
```

Root Cause (Programming Perspective)

Counter stored per-username, increment deferred until after slow password check. N parallel requests = N reads before first write, so effective limit becomes “N attempts per race window” instead of 3 total.

5.3 Pattern 3: Multi-Endpoint Race

Pattern: Add-to-cart vs checkout

1. Identify two endpoints touching same shared state
2. Build two Repeater groups (add item + checkout)
3. Send both groups in parallel repeatedly
4. Check cart/response after each burst
5. Item added during race window rides into finalized order

Root Cause Pattern

```
if cart.total <= user.balance: # validation step (time T0)
    # --- race window: cart is still mutable here ---
    finalize_order(cart) # commit step (time T0 + delta)
```

Root Cause (Programming Perspective)

Validation and finalization are two separate steps (sometimes separate HTTP requests). Cart can be mutated *after* balance check passed but *before* order confirmed. Any item added during that gap rides into finalized order without being balance-checked.

5.4 Pattern 4: Single-Endpoint Race

Pattern: Email change with two addresses

1. Duplicate email-change request with different email values
2. Send both requests **in parallel** under same session
3. One request's data may "win" the write race
4. Other request's confirmation token may still be generated
5. Click confirmation link — address changes to target

Root Cause Pattern

```
session['pending_email'] = new_email # write 1 (request A)
session['pending_email'] = new_email # write 2 (request B, races A)
send_confirmation(session['pending_email']) # read —which write "won"?
```

Root Cause (Programming Perspective)

Both requests operate on same session-scoped variable with no locking. Final value depends on which write executed last, but confirmation token/email pairing may have captured the other request's data, producing mismatch attacker can exploit.

5.5 Pattern 5: Partial-Construction Race

Pattern: Confirm account before token stored

1. Study registration flow: user row created, then token stored
2. Probe /confirm with edge-case values:
 - No token → Missing parameter
 - Empty token → Forbidden (patched)
 - Empty array token[]= → Incorrect token: Array (accepted!)
3. Registration is slower than confirmation (benchmark)
4. Add deliberate delay to confirmation requests
5. Flood confirmation attempts while registering many users
6. Watch for Account registration successful

Root Cause Pattern

```
user = db.insert_user(username, email, password) # write 1: row exists, token
NULL
token = generate_token()
db.update_user_token(user.id, token) # write 2: token finally set
```

Root Cause (Programming Perspective)

Between write 1 and write 2 the token column is NULL. PHP-style loose comparison can make `array` submitted as `token[]=` evaluate as equal to uninitialized/NULL stored token. Landing confirmation in that gap confirms account without knowing real token.

5.6 Pattern 6: Time-Sensitive Tokens

Pattern: Predictable password reset token

1. Trigger password reset — observe token format
2. Token is same length but different each time
3. Send two reset requests under different sessions in parallel
4. If response times match, both emails contain **same token**
5. Token is timestamp-based, not cryptographically secure
6. Use your token, edit username parameter to target
7. Reset target's password, log in

Root Cause Pattern

```
token = md5(username_secret_salt + str(time.time()))
```

Root Cause (Programming Perspective)

Token generation relies on high-resolution timestamp instead of CSPRNG. If two reset requests processed at same server timestamp (achieved by racing under separate sessions to dodge PHP's per-session request lock), they hash to identical token regardless of which username triggered them.

6 Phase 5: Impact Confirmation

Checklist

- ✓ **Coupon abuse:** Refresh cart, verify discount applied multiple times
- ✓ **Rate-limit bypass:** Check for 302 redirect (successful login)
- ✓ **Multi-endpoint:** Check cart state, order confirmation, balance
- ✓ **Single-endpoint:** Verify email changed, token worked
- ✓ **Partial construction:** Log in with registered account
- ✓ **Time-sensitive:** Reset target password, log in
- ✓ **Out-of-band:** Check inbox, refresh cart, recheck balance
- ✓ **Don't trust status codes alone** — a collision doesn't always produce an obviously different response

7 Complete Testing Checklist

Checklist**Phase 1: Predict Potential Collisions**

- ☐ Map all endpoints that touch shared state (sessions, database rows, counters)
- ☐ Identify security-critical endpoints (coupons, balances, login, password reset)
- ☐ Look for collision potential: same session, same resource, same counter
- ☐ Note any "one-time" actions or multi-step workflows

Phase 2: Probe for Clues

- ☐ Perform action normally once, note response and timing
- ☐ Repeat and compare token/value changes
- ☐ Send two requests sequentially in Repeater, note response times
- ☐ Test with malformed/edge-case parameter values
- ☐ Check for session-based locking masking the race

Phase 3: Choose Timing Technique

- ☐ HTTP/2 available? → Single-packet attack
- ☐ HTTP/1.1 only? → Last-byte synchronization
- ☐ Session locking masking? → Use two separate sessions
- ☐ Need controlled delay? → Turbo Intruder with gate control

Phase 4: Exploitation

- ☐ **Limit-Overrun:** Send same endpoint repeated in parallel
- ☐ **Rate-Limit Bypass:** Fire guesses in one race window
- ☐ **Multi-Endpoint:** Race two different endpoints touching same record
- ☐ **Single-Endpoint:** Race same endpoint with different parameter values
- ☐ **Partial-Construction:** Flood confirm requests during registration
- ☐ **Time-Sensitive:** Race requests to generate matching tokens

Phase 5: Confirmation

- ☐ Verify out-of-band (inbox, cart state, balance, admin panel)
- ☐ Don't trust status codes alone
- ☐ Retry with reordered request sequencing
- ☐ Warm connection with throwaway request first
- ☐ Document exact payload and conditions

8 Turbo Intruder Templates

8.1 Template 1: Single Request Repeated

```
def queueRequests(target, wordlists):
    engine = RequestEngine(endpoint=target.endpoint,
                           concurrentConnections=30,
                           requestsPerConnection=30,
                           pipeline=False)

    for i in range(30):
        engine.queue(target.req, i)
        engine.queue(target.req, target.baseInput, gate='race1')
    engine.start(timeout=5)
    engine.openGate('race1')
    engine.complete(timeout=60)

def handleResponse(req, interesting):
    table.add(req)
```

8.2 Template 2: Two Different Requests Racing

```
def queueRequests(target, wordlists):
    engine = RequestEngine(endpoint=target.endpoint,
                           concurrentConnections=30,
                           requestsPerConnection=100,
                           pipeline=False)

    request1 = '''
POST /target-URI-1 HTTP/1.1
Host: <TARGET>
Cookie: session=<SESSION>

parameter=value
'''
    request2 = '''
GET /target-URI-2 HTTP/1.1
Host: <TARGET>
Cookie: session=<SESSION>
'''

    engine.queue(request1, gate='race1')
    for i in range(30):
        engine.queue(request2, gate='race1')
    engine.openGate('race1')
    engine.complete(timeout=60)

def handleResponse(req, interesting):
    table.add(req)
```

8.3 Template 3: Single-Packet Attack (HTTP/2)

```
def queueRequests(target, wordlists):
    engine = RequestEngine(endpoint=target.endpoint,
                           concurrentConnections=1,
                           engine=Engine.BURP2) # single-packet attack
    for word in open('/path/to/wordlist.txt'):
        engine.queue(target.req, word.rstrip(), gate='race1')
    engine.openGate('race1')

def handleResponse(req, interesting):
    table.add(req)
```

9 Tools Reference

Tool	Purpose
Burp Suite Repeater (2023.9+)	Send group in parallel (single-packet attack)
Turbo Intruder	Fine-tuned timing control, scriptable
Burp Collaborator	Out-of-band verification
Burp Inspector	Decode/analyze tokens
PayloadsAllTheThings	Reference payloads

10 Common Pitfalls & How to Avoid Them

- **Pitfall:** Testing only one endpoint, missing multi-endpoint races
 - **Solution:** Map all endpoints touching the same shared state

- **Pitfall:** Assuming session locking means no race is possible
 - **Solution:** Retest with two separate sessions
- **Pitfall:** Not checking for partial-construction sub-states
 - **Solution:** Fuzz token-comparison endpoints with edge-case values
- **Pitfall:** Only testing same-session races
 - **Solution:** Test cross-session races where different users share a resource
- **Pitfall:** Trusting status codes alone for confirmation
 - **Solution:** Verify out-of-band (inbox, cart, balance)
- **Pitfall:** Giving up after one failed burst
 - **Solution:** Retry with reordered sequencing, connection warming, delays
- **Pitfall:** Not benchmarking relative response times
 - **Solution:** Add deliberate delay to faster requests

11 Prevention Recommendations

- **Atomic check-and-act:** Wrap read, decision, and write in single DB transaction
- **Row-level locking:** Use `SELECT ... FOR UPDATE` around critical sections
- **Idempotency keys:** Unique constraints so second insert fails
- **Avoid session-level ambiguity:** Don't share one mutable variable across operations
- **CSPRNG tokens:** Never use timestamps for security tokens
- **Rate-limit at infrastructure layer:** WAF/gateway-level limiter
- **Avoid mutable sub-states:** Validate and finalize in one atomic operation

12 References

- PortSwigger Web Security Academy: <https://portswigger.net/web-security/race-conditions>
- PortSwigger Research — *Smashing the State Machine: The True Potential of Web Race Conditions*, James Kettle, Black Hat USA 2023
- PayloadsAllTheThings — Race Condition: <https://github.com/swisskyrepo/PayloadsAllTheThings#race-condition>
- Turbo Intruder: <https://github.com/PortSwigger/turbo-intruder>
- Josip Franjkovic — *Race conditions on the web* (2016)
- Laxman Muthiyah — *Instagram Password Reset Mechanism Race Condition*